

Python OOP Assignment

Amazon-Style Office Order Processing System

Topic: Class, Object, Methods, and Multi-Class Linking

Difficulty Level	Beginner Plus - slightly tougher than yesterday
Submission Type	Python file or Google Colab notebook + output screenshot
Solution	Do not copy from internet. Build step by step using yesterday class concepts.
Main Skill Tested	Connecting multiple classes together like a small office application

1. Business Scenario

Imagine you are working as a junior Python developer in a small e-commerce office. The company sells products online. Customers place orders, the system checks stock, calculates bill amount, updates stock, and shows order details.

Your task is to build a small version of this system using Python OOP. **This is not a full production project.** It is a practice assignment to understand how real office software connects different parts like customer, product, order, and invoice.

2. Concepts You Must Use

- Class and object
- Constructor using `__init__`
- Attributes to store data
- Methods to perform actions
- One class object passed into another class
- Basic validation using if conditions
- Clear output printing for business users

3. Main Requirement

Create an office order processing system with the following 4 classes:

- Customer
- Product
- Order
- Invoice

The important part of this assignment is linking classes. The Order class must use Customer and Product objects. The Invoice class must use the Order object.

4. Class 1 - Customer

Create a class named **Customer**.

Customer Details to Store

- customer_id
- name
- email
- phone
- city

Customer Methods Required

- **display_customer_details()** - Print all customer details clearly.
- **update_phone(new_phone)** - Update the customer phone number.
- **update_city(new_city)** - Update the customer city.

5. Class 2 - Product

Create a class named **Product**.

Product Details to Store

- product_id
- product_name
- category
- price
- stock_quantity

Product Methods Required

- **display_product_details()** - Print all product details clearly.
- **update_price(new_price)** - Update product price only if the new price is greater than 0.
- **reduce_stock(quantity)** - Reduce stock after order is placed.
- **is_stock_available(quantity)** - Return True if enough stock is available. Otherwise return False.

Important Product Rules

- Product price should not be 0 or negative.
- Stock should not become negative.
- If ordered quantity is more than available stock, order should not be placed.

6. Class 3 - Order

Create a class named **Order**. This is the most important class in this assignment because it connects Customer and Product.

Order Details to Store

- order_id

- customer object
- product object
- quantity
- total_amount
- order_status

Order Methods Required

- **calculate_total()** - Calculate total amount using product price * quantity.
- **place_order()** - Check stock availability. If stock is available, reduce stock and mark status as Order Placed. If not, mark status as Order Failed.
- **display_order_details()** - Print customer name, product name, quantity, total amount, and order status.

Important Order Rules

- Order should be placed only if enough stock is available.
- If order is placed, product stock should reduce automatically.
- If stock is not available, do not reduce stock.
- Order status should clearly show whether the order is placed or failed.

7. Class 4 - Invoice

Create a class named **Invoice**. This class should use the Order object.

Invoice Details to Store

- invoice_id
- order object
- gst_percentage
- gst_amount
- final_amount

Invoice Methods Required

- **calculate_gst()** - Calculate GST amount from the order total.
- **calculate_final_amount()** - Add GST amount to total amount.
- **display_invoice()** - Print invoice details clearly for the customer.

Important Invoice Rule

- Invoice should be generated only for successful orders.
- If order failed because of insufficient stock, invoice should not be generated.

8. Required Object Creation

In your final program, create at least:

- 2 customer objects
- 4 product objects

- 3 order objects
- 2 invoice objects for successful orders

9. Required Test Scenarios

Your program must test the following situations:

Scenario	What Should Happen	Example
Successful order	Stock should reduce and invoice can be generated.	Laptop stock 10, order quantity 2
Failed order	Order should fail and stock should not reduce.	Mouse stock 3, order quantity 8
Phone update	Customer phone should change.	Old phone to new phone
Price update	Product price should update only if valid.	50000 to 48000
Invalid price	System should reject negative or zero price.	-100 or 0

10. Expected Program Flow

1. Create customers.
2. Create products.
3. Display available product details.
4. Place a successful order.
5. Display order details.
6. Generate invoice for successful order.
7. Display updated product stock.
8. Try placing an order with quantity more than available stock.
9. Show order failed message.
10. Do not generate invoice for failed order.

11. Expected Output Format

Your output does not need to match this exactly, but it should be clear and easy to read.

Customer Details
 Customer ID: C001
 Name: Ravi
 Email: ravi@gmail.com
 Phone: 9876543210
 City: Chennai

Product Details
 Product ID: P001
 Product Name: Laptop
 Category: Electronics
 Price: 50000

Stock: 10

Order Details

Order ID: O001

Customer Name: Ravi

Product Name: Laptop

Quantity: 2

Total Amount: 100000

Order Status: Order Placed

Invoice Details

Invoice ID: INV001

Order ID: O001

Customer Name: Ravi

Product Name: Laptop

Total Amount: 100000

GST Amount: 18000

Final Amount: 118000

Updated Product Stock

Laptop Stock: 8

Failed Order Example

Order ID: O002

Product Name: Mouse

Requested Quantity: 8

Available Stock: 3

Order Status: Order Failed - Not enough stock available

12. Student Submission Checklist

- All 4 classes are created: Customer, Product, Order, Invoice.
- Objects are created from each class.
- Order class connects Customer and Product objects.
- Invoice class connects Order object.
- Stock reduces after successful order.
- Stock does not reduce after failed order.
- Invoice is generated only for successful order.
- Output is clear and readable.
- Code is written with proper method names.
- A short explanation is added at the end.

13. Short Explanation to Submit

Along with the code, write a short explanation answering these questions:

- What classes did you create?

- What objects did you create?
- How does the Order class connect Customer and Product?
- How does the Invoice class connect with Order?
- What happens when stock is available?
- What happens when stock is not available?

14. Marking Criteria

Criteria	Marks
Customer class and methods	10
Product class and methods	15
Order class linking customer and product	20
Stock validation and stock update	15
Invoice class linked with order	15
Successful and failed order scenarios tested	10
Clear output and readable code	10
Short explanation submitted	5
Total	100

15. Important Reminder

Do not search for a full ready-made solution. Break the problem into small parts. First create Customer. Then create Product. Then connect them using Order. Finally connect Order with Invoice.

Main idea to remember: Customer buys Product through Order. Invoice is created from Order.

End of Assignment